

EDA_EMG OTA 正式发布说明

更新时间：2026-09-08 · 目标读者：参与 EDA_EMG 固件发布 / 升级的同事

仓库：EDA_EMG_ESP32 (ESP32 端)、EDA_EMG (STM32 端)

一句话：改版本号 → 编译 → 打包 → 传云端 → 设备联网升级，全程有脚本和版本号统一口径。

0. 总览：一次正式发布要做什么

1. 确认 **测试开关已关闭**（正式版走版本比较，不强制升级）。
2. 给 **三个目标**（ESP32 / 图片 / STM32）中本次改动的 bump 版本号。
3. **编译**出固件（ESP32 用 `idf.py build`；STM32 若改版本号要在 IAR 重新编译）。
4. **打包**：跑脚本生成 `eda_emg/` 目录 + `version.json`。
5. **上传**：把整个 `eda_emg/` 目录推到正式云端。
6. **设备验证**：设备联网进 OTA 页，确认只对“云端版本 > 本地版本”的目标提示更新并升级成功。

设备侧判定规则：**仅当云端版本 > 本地版本** 才提示可升级；升级后两者相等，不再重复提示。所以“要发新版”=“改小对应固件的版本号再重新打包上传”。

1. 发布前必查：测试开关

位于 `main/ota/app_ota.c` 第 46 行附近：

```
#define OTA_TEST_FORCE_UPGRADE 0
```

开关值	行为	适用场景
1（测试）	跳过版本比较，只要拉到清单就把三目标全部标记“有更新”，强制完整升级	本地联调、上板验证升级链路

0 (正式)	走版本比较, 仅当 云端版本 > 本地版本 才升级	正式发布必须 0
--------	---------------------------	----------

⚠ 正式发布前, 务必确认 `OTA_TEST_FORCE_UPGRADE` 为 0。否则所有已是最新版的设备也会被强制重刷。

2. 三个目标的版本号 (口径必须统一)

版本号统一用 **两位** 主.次 (内部 16 位 `0xMMmm` : 高字节主、低字节次)。三处独立维护, 只 bump 本次真正改动的目标。

目标	版本号定义位置	当前值	云端校验方式
ESP32 固件	main/ota/app_ota.h 里的 APP_FW_VERSION	0x0200 (= 2.0)	sha256
图片资源 (SPIFFS)	spiffs_data/image_ver.txt (内容形如 1.0)	1.0	sha256
STM32 固件	D:\GitLab\EDA_EMG\code\MODBUS\MB_RegMap.h 里的 FW_VERSION	0x0100 (= 1.0)	crc32

注意区分 STM32 头文件里的两个宏: `MB_REG_FW_VERSION` (寄存器地址, 勿动) 与 `FW_VERSION` (实际版本号, 发布时改它)。

版本号 0xMMmm 规则

- 高字节 = 主版本, 低字节 = 次版本; 显示为 主.次。
- 例: `0x0201` → 显示 2.1 ; `0x0100` → 1.0。
- ESP32 版本号会被打包脚本从 `app_ota.h` 自动提取, 不会写错——改源码即可。

3. 编译

3.1 ESP32 固件

```
cd D:\GitLab\EDA_EMG_ESP32
idf.py build
```

- 产出: `build/eda_emg_esp32.bin` (OTA 用) 与 `build/storage_a.bin` (图片资源)。
- 团队统一使用 **ESP-IDF v5.5.3** 工具链编译。

3.2 STM32 固件

- 若本次改了 `FW_VERSION` , 需在 **IAR** 重新编译。
- 产出: `D:\GitLab\EDA_EMG\EWARM\EDA_EMG\Exe\EDA_EMG.hex` (打包脚本会转 bin) 。

4. 打包

在 `EDA_EMG_ESP32` 根目录执行:

```
py tools/build_ota_package.py
```

脚本仅依赖 Python 标准库, 会自动:

- 从源码自动提取版本号 (`APP_FW_VERSION` / `image_ver.txt` / `FW_VERSION`) , 保证设备自报版本与云端一致。
- 生成 `eda_emg/` 目录与 `version.json` (含 `sha256` / `crc32` / `size`) 。
- **自动处理 STM32 目标**: 电脑上有 `EDA_EMG` 仓库且已编译出 hex, 就重新打包 `stm32/` 目标 (覆盖) ; 没有 STM32 源码时, 若 `eda_emg/` 里**已经存在**上次生成好的 `stm32/fw.bin` 与 `version.json` 里的 `stm32` 条目, 就**原样沿用、不覆盖丢失**; 两者都没有才省略 `stm32` 项。设备端会自动跳过缺失的目标, 不影响升级。

同事电脑通常**没有 STM32 工程** (`D:\GitLab\EDA_EMG` 不存在) , 直接跑 `py tools/build_ota_package.py` 即可: 脚本会**沿用已有的 stm32 信息** (打印"沿用已有目标") , 不会因为重跑就删掉原来 json 里已经有的 `stm32` 数据。需要发布新的 STM32 固件时, 才在有 STM32 工程并编译好的机器上跑。

输出目录结构 (有 STM32 时) :

```
eda_emg/
├─ version.json
├─ esp32/eda_emg_esp32.bin
├─ image/storage_a.bin
└─ stm32/fw.bin
```

同事电脑无 STM32 工程、但 `eda_emg/` 里已有旧 `stm32/` 目标时，`stm32` 条目会原样保留；只有从没有过 STM32 目标才会省略它。

三个目标版本号可独立指定（默认都按当前源码提取），需要临时指定时在终端加参数：

```
py tools/build_ota_package.py --esp32-ver 0.1.0 --image-ver 0.1.0 --stm32-ver 0.1.0
```

5. 上传云端

正式云端地址（设备 OTA 读此清单）：

```
http://health.eda-
global.cn/pelvicFloorMuscleUpgradeKit/fota/eda_emg/version.json
```

- **正式云端没有 FTP**，需手动上传整个 `eda_emg/` 目录到上述路径（覆盖 `version.json` 及 `esp32/` `image/` `stm32/` 子目录）。
- 脚本里的 `--upload` 走的是旧测试服务器 FTP，仅联调时用，**不是**正式线路。

⚠ 上传是覆盖生产云端，确认版本号与打包结果无误再传。

6. 设备验证流程

1. 设备联网，进入 OTA 升级页，点“检查更新”。
2. 确认只有**本次已 bump 版本的目标**显示“有更新”（未改的目标应显示“已最新”）。
3. 点升级，观察进度条；升级顺序为 **STM32** → **图片** → **ESP32**。
4. 全部完成后设备重启（图片切槽、ESP32 切 boot 分区、STM32 跳新 APP）。

5. 再进 OTA 页检查一次，应显示"全为最新"，不再提示更新。

7. 常见坑（务必看）

#	现象 / 坑	处理
1	git pull 后编译配置没生效（尤其内存相关配置）	删除本地根目录 sdkconfig 后再 build，IDF 会以 sdkconfig.defaults 重新生成
2	同事用不同 ESP-IDF 版本编译，导致 dependencies.lock / 行为不一致	团队统一 ESP-IDF v5.5.3
3	只烧了 app-flash，图片资源（SPIFFS）缺失	app-flash 不含 storage 资源，需 全量 flash （app + storage 一起烧）
4	OTA 页面字库不对 / 缺字	重新生成 OTA 专用字库，见附录 8.2

8. 附录

8.1 测试开关与版本比较逻辑

- OTA_TEST_FORCE_UPGRADE = 1：强制全部升级（联调用），发布前必须改回 0。
- = 0：app_ota_ver_compare(cloud, local) > 0 才升级。

8.2 重新生成 OTA 专用字库

OTA 升级页使用 LVGL 子集字库 spiffs_data/png/font/lv_font_OTA.bin，若页面改文案出现缺字需重新生成：

```
node tools/gen_ota_font.js
```

首次跑会失败的常见原因：脚本 tools/gen_ota_font.js 第 11 行左右，FONT_CONV 被硬编码成了作者本机路径（形如 C:/Users/lizhi/AppData/Local/npm-cache/_npx/.../lv_font_conv.js）。

处理：改成**你自己机器上** `lv_font_conv` 的**全局路径**（先安装 Node.js 与 `lv_font_conv`，用 `npm root -g / which lv_font_conv` 找到它）。

脚本关键参数：源字体 `C:/Windows/Fonts/simhei.ttf`；输出

`spiffs_data/png/font/lv_font_OTA.bin`；`--size 20 --bpp 4 --format bin -r 0x20-0x7F`。